

OnionMonad: Onion Circuits as Client-Sealed Loops of Registered Operation Symbols

Ryan J. Kung

Abstract

The Rings onion data plane interprets two layers, relay and exit, and answers along the reversed path through relays that keep per-circuit return state and forward one client-bound ciphertext byte-identical. We generalise the circuit to a term over a static signature Σ of operation symbols: nodes are partial Σ -algebras that register symbols, never applications; a circuit is a client-sealed loop $\diamond \rightarrow g \rightarrow \dots \rightarrow g \rightarrow \diamond$ that evaluates one application per hop and composes them in the Kleisli category of the hop effect monad, with **relay** as the identity. Every hop parses one uniform layer; values travel in a fixed-width carry slot enciphered with AEZ, one length-preserving layer per relay and one authenticated layer per consumer; replies to world-facing sessions ride single-use reply blocks under credit flow control. We state ten laws, among them noninterference over hop views, client path authority (which admits Arrow and Selective composition and excludes monadic bind), byte unlinkability, and single-use admission, list the residual leakage, and map each law to a test obligation.

I. Introduction

Today's circuit is the term **relay**^k $\ggg$ (*svc*, *body*) evaluated on the forward path and answered on its reverse (§X). This paper states the general structure of which it is the degenerate instance, as definitions D1–D9 and laws L1–L10 of issue #834, numbered identically. Design lineage: Sphinx headers and wide-block payloads [1], reply blocks [2], [3], loops [4], AEZ [5], Kleisli semantics [6], generic effects [7], [8], and the Arrow/Selective hierarchy [9]–[11].

II. Signature and Algebras

Definition D1 (signature). Σ is a finite, statically defined table of symbols; each $f \in \Sigma$ has

$$\text{spec}(f) = (\text{name}, \text{Args}_f, \text{In}_f, \text{Out}_f, \\ W(f), L(f), \text{pos}(f), \text{quota}(f))$$

with *name* identifying the backend (e.g. `jev.typesafe`), Args_f client constants, $\text{In}_f, \text{Out}_f$ carried types, $W(f) \in \mathbb{N}$ the exact output width in bytes, $L(f)$ the exact latency class, $\text{pos}(f) \subseteq \{\text{Intermediate}, \text{WorldFacing}\}$, and $\text{quota}(f)$ the admission budget. W, L are protocol data. The identity symbol is

$$\text{relay} : X \rightarrow X, \quad W(\text{relay}) = |X|, \quad L(\text{relay}) = 0.$$

Definition D2 (registration). Node n is a partial Σ -algebra: a set $\Sigma_n \subseteq \Sigma$ and, for each $f \in \Sigma_n$, a generic effect [7]

$$\llbracket f \rrbracket_n : \text{Args}_f \times \text{In}_f \rightarrow M \text{Out}_f,$$

Ryan J. Kung can be reached at ryankung@ieee.org. Manuscript created September 24, 2026. This document freezes the design of Rings issue #834 (rings-node, module onion) before its Phase 2 code; it is a specification, not shipped behaviour. The segment relay count s (`ONION_SEGMENT_RELAYS`) is left symbolic.

M the hop effect monad (failure, delay, world I/O). n publishes $f \in \Sigma_n$, never an application. **relay** is published in `OnlineNodeDescriptor::capabilities`; every other f in one signed descriptor per symbol carrying n 's invocation epoch and $\text{quota}(f)$.

A node is a handler [8] for the operations it registers; **WorldFacing** symbols (`tcp`, `https`) may hold a session, **Intermediate** symbols are one-shot.

Definition D3 (application). An application is $(f, \bar{a})$, $\bar{a} \in \text{Args}_f$, typed $(f, \bar{a}) : \text{In}_f \rightsquigarrow \text{Out}_f$; it is sealed in the layer of the hop that evaluates it and visible to no other hop.

III. Pipelines

Definition D4a (pipeline). Pipelines are the morphisms of the free category $\mathcal{F}(\Sigma)$ on the graph with types as vertices and applications as edges; composition is

$$\frac{\alpha : X \rightsquigarrow Y \quad \beta : Y \rightsquigarrow Z}{\alpha \ggg \beta : X \rightsquigarrow Z}.$$

$P = (f_1, \bar{a}_1) \ggg \dots \ggg (f_n, \bar{a}_n)$ is well formed iff

$$n \geq 1, \quad \forall k < n : \text{Out}_{f_k} = \text{In}_{f_{k+1}}, \\ \forall k < n : \text{Intermediate} \in \text{pos}(f_k),$$

f_n is a **WorldFacing** session symbol $\Rightarrow n = 1$.

Violations are rejected at construction.

Definition D4b (Kleisli denotation). For $\phi : X \rightarrow MY$, $\psi : Y \rightarrow MZ$ in $\mathbf{KI}(M)$ [6], $\phi \ggg \psi = \mu_Z \circ M\psi \circ \phi$. For hop assignment $h_1..h_n$ with $f_k \in \Sigma_{h_k}$,

$$\llbracket P \rrbracket(v_0) = (\llbracket f_1 \rrbracket_{h_1}(\bar{a}_1, -) \ggg \dots \ggg \llbracket f_n \rrbracket_{h_n}(\bar{a}_n, -))(v_0),$$

the unique functor $\mathcal{F}(\Sigma) \rightarrow \mathbf{KI}(M)$ extending the hop interpretations. The client consumes $\llbracket P \rrbracket(v_0)$.

Law L1 (identity). $\llbracket \text{relay} \rrbracket = \eta$, hence $\text{relay} \ggg f = f = f \ggg \text{relay}$ on values. The loop builder factors through nf , which deletes relay applications: $\text{Loop}(P) = \text{Loop}(\text{nf}P)$, so $\text{Loop}(\text{relay} \ggg P) \stackrel{d}{=} \text{Loop}(P)$ and every hop's view is unchanged.

Law L2 (associativity). $(f \ggg g) \ggg h = f \ggg (g \ggg h)$ on values; immediate from the monad laws, and $\mathcal{F}(\Sigma)$ has no bracketing.

Phase 1 admits only $\text{relay}^k \ggg (f, \bar{a})$ with $\text{WorldFacing} \in \text{pos}(f)$, which is today's circuit (§X).

IV. The Loop

Definition D4c, D5 (loop, segment). Let $\diamond$ be the client, g its entry guard, $s \geq 1$ the protocol constant $\text{ONION_SEGMENT_RELAYS}$. Segment $k \in [0, n]$ is $r_{k,1}..r_{k,s}$ from producer p_k to consumer c_k ,

$$\begin{aligned} p_0 &= \diamond, & p_k &= h_k, & c_k &= h_{k+1}, & c_n &= \diamond, \\ \text{Loop}(P) : \diamond &\rightarrow r_{0,1..s} \rightarrow h_1 \rightarrow r_{1,1..s} \rightarrow \dots \\ &\rightarrow h_n \rightarrow r_{n,1..s} \rightarrow \diamond. \end{aligned}$$

Positions are $i \in [1, N]$, $N = n + (n+1)s$; position i evaluates f_i , which is relay on every $r_{k,j}$. $\text{Loop}(\text{nf}P)$ is the symbol word $\text{relay}^s f_1 \text{relay}^s \dots f_n \text{relay}^s$.

Law L7 (guard closure). $r_{0,1} = r_{n,s} = g$, and the other $N-2$ positions carry pairwise distinct DIDs, distinct from g . g is the only hop adjacent to $\diamond$.

L7 requires $s \geq 1$. Route validation admits exactly the guard twice ($\text{has_duplicate_dids}$), checks the loop shape against s , and derives the hop bound from N .

V. Uniform Layer and Timing

Definition D6 (uniform layer). Every position i has one layer shape

$$\lambda_i = (f_i, \bar{a}_i, \text{next}_i, e_i, x_i, \nu_i, \kappa_i),$$

epoch e_i , expiry x_i , replay nonce ν_i , and carry keys $\kappa_i = (k_i^{\text{in}}; k_{i,1..s+1}^{\text{out}})$ of constant length; a relay has $\bar{a}_i = ()$ and random k^{out} . The header is Sphinx-shaped [1]:

$$\text{peel}(sk_i, H_i) = (\lambda_i, H_{i+1}), \quad |H_i| = |H|.$$

The cell on edge $i \rightarrow \text{next}_i$ is $(H_{i+1}, y_i) \in \{0, 1\}^{|H|} \times \{0, 1\}^C$ for every i .

Law L3 (width). The carry slot has width C on every edge. With plaintext width $C_0 = C - \tau$, $W(f) < C_0$ for all $f \in \Sigma$ and

$$\text{pad}(b) = b \parallel 80 \parallel 00^{C_0 - |b| - 1}, \quad |\text{enc}_f(w)| = W(f);$$

an output with $|\text{enc}_f(w)| > W(f)$ fails closed, never truncated.

Law L4 (timing). $t_i^{\text{dep}} = t_i^{\text{arr}} + L(f_i)$ exactly. An evaluation unfinished at t_i^{dep} is dropped, never released late.

Algorithm 1 $\text{Hop}_i(\text{from}, H, y, t^{\text{arr}})$: one position, all symbols

```

1:  $(\lambda, H') \leftarrow \text{peel}(sk_i, H)$ ; if  $\perp$  then drop
2: if  $t^{\text{arr}} > x \vee e \neq \text{epoch}_i \vee \nu \in R_i$  then
3:   drop ▷ L9
4:  $R_i \leftarrow R_i \cup \{(\nu, x)\}$ 
5: if  $f = \text{relay}$  then
6:    $y' \leftarrow D_{k_{\text{in}}}^0(y)$ 
7: else
8:    $b \leftarrow D_{k_{\text{in}}}^\tau(y)$ ; if  $b = \perp$  then drop ▷ L8
9:    $w \leftarrow \llbracket f \rrbracket(\bar{a}, \text{enc}^{-1} \text{pad}^{-1}(b))$  by  $t^{\text{arr}} + L(f)$  ▷ L4
10:  if  $\text{late} \vee |\text{enc}_f(w)| > W(f)$  then
11:    drop ▷ L3
12:   $y' \leftarrow E_{k_1^{\text{out}}}^0 \circ \dots \circ E_{k_s^{\text{out}}}^0 \circ E_{k_{s+1}^{\text{out}}}^\tau(\text{pad enc}_f(w))$ 
13: release  $(H', y')$  to next at  $t^{\text{arr}} + L(f)$ 

```

VI. Carry

Write E_k^τ, D_k^τ for AEZ v5 [5], [12] encryption and decryption with empty nonce and associated data; $|E_k^\tau(m)| = |m| + \tau$, and D_k^τ returns $\perp$ unless the deciphered block ends in τ zero bytes.

Definition D7 (carry). For segment k the client draws fresh keys $k_{r_{k,1}}, \dots, k_{r_{k,s}}, k_{c_k}$ and places them

$$\kappa_{p_k}^{\text{out}} = (k_{r_{k,1}}, \dots, k_{r_{k,s}}, k_{c_k}), \quad \kappa_{r_{k,j}}^{\text{in}} = k_{r_{k,j}}, \quad \kappa_{c_k}^{\text{in}} = k_{c_k}.$$

With $b_k = \text{pad}(\text{enc}(v_k))$ and $\tau = 16$:

$$\begin{aligned} y_{k,0} &= E_{k_{r_{k,1}}}^0 \circ \dots \circ E_{k_{r_{k,s}}}^0 \circ E_{k_{c_k}}^\tau(b_k), \\ y_{k,j} &= D_{k_{r_{k,j}}}^0(y_{k,j-1}), \quad b_k = D_{k_{c_k}}^\tau(y_{k,s}). \end{aligned}$$

A consumer with $\text{In}_f = ()$ ignores the slot.

Since s is constant, $|\kappa_i|$ is constant and reveals no position. By L9 a layer is admitted once, so every AEZ key enciphers exactly one C -byte value, far below AEZ's birthday bound.

VII. Reply Blocks

Definition D8 (SURB). A single-use reply block for a session symbol at h is the client-sealed loop suffix after h :

$$v = (\text{next}, H_v, k_{r_1}, \dots, k_{r_s}, k_\diamond, x_v).$$

The suffix of the loop that opened or continued the session is itself a SURB; a forward frame may carry σ further SURBs in its carry value.

Invariant 1 (Credit). Let Q_h be h 's unexpired SURBs, $u(t)$ the SURBs received and $r(t)$ the replies emitted by time t . Then

$$r(t) \leq u(t), \quad |Q_h| = 0 \Rightarrow \text{tcp stops reading its socket.}$$

Each reply consumes one v , produced as segment n of D7 with $p_n = h$; replenishment resumes reading.

VIII. Security Laws

Assumption 1 (Primitives). Under a key it does not hold, a hop sees E_k^0 as a uniformly random permutation of $\{0, 1\}^C$ and E_k^r as robust AE [5]; the header hides every layer but the hop's own [1]. Network delays δ are fixed per edge.

Definition D9 (view). Over client coins ρ (keys, nonces, header randomness) and delays δ ,

$$\text{view}_i = (f_i, \bar{a}_i, v_{i-1}, \text{from}_i, \text{next}_i, e_i, x_i, \\ \text{bucket}, t_i^{\text{arr}}, t_i^{\text{dep}}, H_i, y_{i-1}, H_{i+1}, y_i),$$

with $v_{i-1} = \perp$ when $f_i = \text{relay}$ (a relay consumes no value).

Law L5 (noninterference). For loops ℓ, ℓ' let

$$\pi_i(\ell) = (f_i, \bar{a}_i, v_{i-1}, \text{from}_i, \text{next}_i, e_i, x_i), \\ \sigma(\ell) = f_1 \cdots f_N,$$

$$\pi_i(\ell) = \pi_i(\ell') \wedge \sigma(\ell) = \sigma(\ell') \Rightarrow \text{view}_i(\ell) \stackrel{d}{=} \text{view}_i(\ell').$$

Symbols may leak; $\bar{a}_j, v_j$ for $j \neq i$, other than the consumed v_{i-1} , may not [13].

Proposition 1 (L5 from D6, D7, L3, L4). Under Assumption 1, every loop built by D4c–D7 satisfies L5.

Proof. Lengths: $|H_i| = |H|$, $|y_i| = C$ (D6, L3). Times: $t_i^{\text{arr}} = \sum_{j < i} (L(f_j) + \delta_j)$ by L4, a function of σ, δ . Headers: H_i, H_{i+1} reveal only λ_i , fixed by π_i up to fresh coins. Carry: if $i = r_{k,j}$, then $D_{k_{r_{k,j}}}^0(y_{k,j-1})$ is an E output under $k_{r_{k,j+1}}$ or k_{c_k} , a fresh key not held by i , hence uniform on $\{0, 1\}^C$ and independent of v_k . If $i = c_k$, it learns v_{i-1} , which π_i fixes. If $i = p_k$, its output is computed from v_{i-1} and $\bar{a}_i$. No component depends on $\bar{a}_j, v_j$ otherwise. $\square$

Law L6 (path authority). next_i is a field of λ_i , fixed by $\diamond$ at construction: next_i is independent of every v_j . No symbol chooses a hop.

Proposition 2 (Admissible combinators). L6 admits Arrow composition [9] and Selective branching [11] over client-sealed branches, and excludes monadic bind.

Proof. $\gg$ and $\text{select} : F(A + B) \rightarrow F(A \rightarrow B) \rightarrow F B$ fix the set of possible effects before any value exists, so $\diamond$ can seal every hop of every branch in advance. Bind $m \gg k, k : A \rightarrow M B$, chooses the continuation, and so its hops, from a run-time value: either a hop constructs next (violating L6) or $\diamond$ seals $|A|$ continuations. This is the arrows-meticulous, monads-promiscuous separation of [10]. $\square$

“OnionMonad” names M , the effect inside one hop; across hops only the static fragment of $\mathbf{KI}(M)$ is exposed.

Law L8 (unlinkability). On consecutive edges $H_i \neq H_{i+1}$ and $y_{i-1} \neq y_i$ (one layer peeled, one AEZ layer removed per hop). If any $y_{k,j}$ is modified to $\tilde{y} \neq y_{k,j}$,

$$\Pr[D_{k_{c_k}}^\tau(\tilde{y}_{k,s}) \neq \perp] \leq 2^{-8\tau} + \epsilon_{\text{AEZ}},$$

and a rejected value yields nothing; no bit of a tag survives.

Table I
Residual leakage, stated and accepted

leakage	bound
symbols σ are public;	L5 hypothesis
λ_i names f_i	
a WorldFacing hop knows it faces the world	no other hop learns whether symbols follow (L3, D6)
g sees departure and return	loop round-trip time
latency class	L4 quantises timing; cannot remove it
hosted backend	sees $\bar{a}$ of every application of its symbol
p_k holds $\kappa_{p_k}^{\text{out}}$	p_k colluding with $r_{k,j}$ links two positions of one segment; no value leaks, and timing links them anyway
client-authored semantics ($\bar{a}_{i+1}$ depends on meaning of v_i)	out of protocol scope

Under Assumption 1 consecutive y are independent uniform strings to any hop lacking both keys. A modification propagates through every later wide-block decipherment, so, unlike counter-mode relay cryptography [14], a flipped bit is never observed in place; this is Sphinx's wide-block payload [1], [15] with AEZ in place of LIONESS.

Law L9 (replay). Every hop admits each ν at most once within $[\cdot, x]$; R_i holds (ν, x) until x . A SURB is a layer, hence single use.

Law L10 (wire). Phase 1 preserves today's bytes (golden tests); Phase 2 replaces them with a new pinned golden set in one cutover, without versioning.

IX. Accepted Leakage

Table I is exhaustive with respect to L5: every other component of view_i is determined by π_i and σ .

X. The Current Design

Proposition 3 (Backward linkability). In circuit/crypto.rs, `encrypt_client_payload_at_sequence` seals the exit's answer once, $c = \text{AEAD}_{pk_\diamond}(\cdot)$, and in circuit/reducer.rs, `advance_backward` re-emits it as `payload = frame.payload`, rewriting only the edge circuit id and the outer hop seal. Every backward relay therefore sees the same c : any two colluding relays, and the exit, link their positions by equality, independently of route length.

The stored return edge $(id', \text{next}) \mapsto (\text{from}, id, pk)$ is the only reason relays hold circuit state; the loop removes it with `Backward`, `OnionBackwardFrame`, `MAX_ONION_RELAY_CIRCUITS`, and `ONION_RELAY_RETURN_TTL_MS`.

Table II
Current circuit versus OnionMonad

concern	current	here
layers	Relay, Exit	one uniform λ_i (D6)
computation	exit service only	any $f \in \Sigma_n$ (D2)
return path	forward path reversed	fresh relays to g (L7)
relay state	relay_returns, ≤ 1024 , TTL 120s refreshed per cell	replay tags only (L9)
return bytes	one ciphertext, identical per edge	one AEZ layer per edge (L8)
replay	exits (consume_forward_ nonce)	every hop (L9)
session replies	no credit bound	one per SURB (Inv. 1)

Table III
Threats and the law that bounds each

threat	bound
hop learns another's $\bar{a}_j, v_j$	L5 (Prop. 1)
position inference	constant $ H , C, \kappa , s$ (D5–D7, L3)
value-dependent timing	exact release $t^{\text{arr}} + L(f)$ (L4)
colluding relays	per-edge re-encipherment (L8); cf. Prop. 3
match bytes	whole-value rejection, $2^{-8\tau}$ (L8)
tagging	$\text{next}_i \in \lambda_i$ (L6); bind excluded (Prop. 2)
hop-chosen routing	one guard closes the loop (L7)
first/last-hop correlation	
replay of layer or SURB	single admission per ν (L9)
reply	$r(t) \leq u(t)$ (Inv. 1)
amplification	
symbol abuse	per-symbol quota(f) (D1)
oversized result	fail closed at $W(f)$ (L3)

XI. Security

Table III maps each threat to the law that bounds it.

The adversary controls any set of nodes other than $\diamond$, and delivery order; it cannot break Assumption 1. Dropping is always possible and out of scope.

XII. Verification

Every law is discharged against the pure builder and reducer, not a shadow model (Table IV).

XIII. Phases

Table V orders the cutover. Open parameters: s (proposed 1), σ and C , and the AEZ implementation (candidate zears, unaudited, gated by Table IV). Non-goals: bind, Selective branches before Phase 2b, session symbols in intermediate position, concrete symbols, and verification of results.

XIV. Related Work

Tor [16] answers along the circuit it built, as Rings does today; Degabriele and Stam [14] formalise the tagging that

Table IV
Obligations and the test that discharges each

obligation	test or property
L10	golden encodings of layers, wire messages, cells, and descriptor signing data, landed before any refactor; a new pinned set at the Phase 2 cutover
L1	carry through relay is identity; $\text{Loop}(\text{relay} \gg P) \stackrel{d}{=} \text{Loop}(P)$
D4a	$\text{Out} \neq \text{In}$, WorldFacing before the last symbol, empty pipeline: rejected at construction
L3	fixed slot width on every edge; output $> W(f)$ dropped
L4	release at $t^{\text{arr}} + L(f)$ and drop on overrun under an injected clock
L5	property test on the pure builder: vary $\bar{a}_j, v_j, j \neq i$; view _{i} bytes, lengths, and release times equal
L7	guard closure; any other duplicate DID rejected
L8	per-hop byte inequality; one-bit flip rejects the whole value
L9	replayed layer and reused SURB rejected
credit	a tcp session emits no reply without a SURB and resumes after replenishment
AEZ	official AEZ v5 vectors; fuzz/ target for $D \circ E = \text{id}$ and authenticator rejection

Table V
Cutover plan

phase	content
1	golden bytes; Σ table; pipeline term; layer build as a fold over P ; reducer and handler as Σ -algebra; route by symbol sequence. Zero wire change, no bump
2a	uniform layer, loop, stateless relays, AEZ carry, SURBs. Breaking
2b	intermediate symbols: Invoke with release time, quota(f). Same cutover; one version bump on its last PR

its counter-mode relay cryptography admits. Chaum's untraceable return addresses [2] and Mixminion's SURBs [3] are the source of D8; Sphinx [1] gives the fixed-length header and a LIONESS [15] payload whose zero-block check L8 inherits. Loopix [4] routes traffic as loops for cover; here the loop is the circuit. AEZ [5], [12] supplies both the length-preserving relay layer and robust AE in one primitive. On the semantic side, Moggi's Kleisli semantics [6], generic effects and handlers [7], [8], and the Arrow/Selective/Monad hierarchy [9]–[11] fix which compositions a client can seal in advance.

References

- [1] G. Danezis and I. Goldberg, "Sphinx: A compact and provably secure mix format," in Proceedings of the 30th IEEE Symposium on Security and Privacy. IEEE, 2009, pp. 269–282.
- [2] D. L. Chaum, "Untraceable electronic mail, return addresses, and digital pseudonyms," Communications of the ACM, vol. 24, no. 2, pp. 84–90, 1981.

- [3] G. Danezis, R. Dingledine, and N. Mathewson, “Mixminion: Design of a type III anonymous remailer protocol,” in *Proceedings of the 2003 IEEE Symposium on Security and Privacy*. IEEE, 2003, pp. 2–15.
- [4] A. M. Piotrowska, J. Hayes, T. Elahi, S. Meiser, and G. Danezis, “The Loopix anonymity system,” in *Proceedings of the 26th USENIX Security Symposium*. USENIX Association, 2017, pp. 1199–1216.
- [5] V. T. Hoang, T. Krovetz, and P. Rogaway, “Robust authenticated-encryption: AEZ and the problem that it solves,” in *Advances in Cryptology – EUROCRYPT 2015*, ser. *Lecture Notes in Computer Science*, vol. 9056. Springer, 2015, pp. 15–44.
- [6] E. Moggi, “Notions of computation and monads,” *Information and Computation*, vol. 93, no. 1, pp. 55–92, 1991.
- [7] G. Plotkin and J. Power, “Algebraic operations and generic effects,” *Applied Categorical Structures*, vol. 11, no. 1, pp. 69–94, 2003.
- [8] G. Plotkin and M. Pretnar, “Handlers of algebraic effects,” in *Programming Languages and Systems (ESOP 2009)*, ser. *Lecture Notes in Computer Science*, vol. 5502. Springer, 2009, pp. 80–94.
- [9] J. Hughes, “Generalising monads to arrows,” *Science of Computer Programming*, vol. 37, no. 1–3, pp. 67–111, 2000.
- [10] S. Lindley, P. Wadler, and J. Yallop, “Idioms are oblivious, arrows are meticulous, monads are promiscuous,” *Electronic Notes in Theoretical Computer Science*, vol. 229, no. 5, pp. 97–117, 2011.
- [11] A. Mokhov, G. Lukyanov, S. Marlow, and J. Dimino, “Selective applicative functors,” *Proceedings of the ACM on Programming Languages*, vol. 3, no. ICFP, pp. 90:1–90:29, 2019.
- [12] V. T. Hoang, T. Krovetz, and P. Rogaway, “AEZ v5: Authenticated encryption by enciphering,” *CAESAR competition submission*, third round, 2017.
- [13] J. A. Goguen and J. Meseguer, “Security policies and security models,” in *Proceedings of the 1982 IEEE Symposium on Security and Privacy*. IEEE, 1982, pp. 11–20.
- [14] J. P. Degabriele and M. Stam, “Untagging Tor: A formal treatment of onion encryption,” in *Advances in Cryptology – EUROCRYPT 2018*, ser. *Lecture Notes in Computer Science*, vol. 10822. Springer, 2018, pp. 259–293.
- [15] R. Anderson and E. Biham, “Two practical and provably secure block ciphers: BEAR and LION,” in *Fast Software Encryption (FSE 1996)*, ser. *Lecture Notes in Computer Science*, vol. 1039. Springer, 1996, pp. 113–120.
- [16] R. Dingledine, N. Mathewson, and P. Syverson, “Tor: The second-generation onion router,” in *Proceedings of the 13th USENIX Security Symposium*. USENIX Association, 2004, pp. 303–320.